LEARN Journal 3 (Due: November 16, 2026)

L – Look Back

- Managing Technical Bottlenecks and Communication under Review Pressure

E – Elaborate

- As we approached the halfway point of the work term, preparations began for both our technical alpha stress test and the formal Mid-Semester Performance Review. I was tasked with scaling our simulated tavern environment from 3 passive NPCs to 10 dynamically interacting characters who possessed persistent memory states and emotional grudges. Two days prior to our scheduled demo, I ran the integrated multi-agent simulation and encountered severe frame stuttering and UI freezes. The event dispatch pipeline was triggering redundant calculation loops across all 10 agents simultaneously, dragging performance down to an unplayable state. Feeling the pressure of the upcoming evaluation, I panicked and attempted multiple hasty, undocumented hotfixes late into the night. Instead of fixing the loop, my changes corrupted the save-state serialization. Exhausted and facing a broken build hours before our supervisor check-in, I had to decide whether to conceal the issue or acknowledge the roadblock. I chose to contact my supervisor directly, presenting the exact error logs, explaining the architectural root cause, and submitting a structured recovery plan.

A – Analyze

- The crisis was caused by two critical missteps: poor architectural testing earlier in the sprint, and letting deadline anxiety dictate my troubleshooting process. Tightly coupling event dispatchers directly to UI listeners without caching or debouncing created a massive algorithmic bottleneck. More importantly, attempting frantic, unversioned fixes under stress compounded a manageable software bug into a systemic failure. However, my decision to communicate transparently with my supervisor prevented an embarrassing demo failure and allowed us to recalibrate expectations collaboratively.

R – Revised Approach

- With my supervisor's support, I reverted the corrupted codebase to the last stable Git commit. I restructured the simulation architecture using an asynchronous observer pattern with event-throttling, successfully decoupling algorithmic updates from the main rendering thread. To prevent future emergencies, I instituted a personal standard operating procedure: all major architectural stress tests must occur at least five business days before scheduled demos, and no code commits may be pushed under sleep deprivation.

N – New Perspective

- Professional maturity is defined by how one handles setbacks, not just successes. Technical roadblocks are an inevitable reality of complex software engineering, but unmanaged anxiety and silence turn minor technical bugs into operational disasters. Proactive, transparent communication with supervisors and structured, calm problem-solving preserve trust and lead to far more resilient software systems.